



Tree Classification and Segmentation from Aerial Imagery

*Classification, Patch Learning, and Semantic Segmentation
using Deep Learning on the MillionTrees Benchmark*

Tambellini Alan 21203966

Siddique Zahra 28725545

Supervised by: Olivier Schwander
In collaboration with: Jacques Gignoux — IEES Paris

Year 2025–2026

PMIND

Contents

1	Introduction	2
2	Dataset Exploration and Selection	2
2.1	Candidate Datasets	2
2.2	The MillionTrees Benchmark	3
2.3	Data Characteristics	3
2.4	Exploratory Data Analysis	4
3	Patch Dataset Construction	5
3.1	Rationale	5
3.2	Positive Patch Generation	5
3.3	Negative Patch Generation	5
3.4	Dataset Versions and Class Balance	5
3.5	Train/Test Split	6
4	Binary Classification	6
4.1	Evaluation Protocol	6
4.2	CNN from Scratch	6
4.3	Vision Transformer from Scratch	7
4.4	ResNet (Pretrained, Fine-tuned)	7
4.5	DINOv2 with Linear Head	8
4.6	CLIP Zero-Shot Baseline	8
4.7	CoOp: Context Optimization	9
4.8	Classification Results	11
5	Data Augmentation	11
5.1	Motivation	11
5.2	Augmentation Pipeline	12
5.3	Impact of Augmentation	12
6	Semantic Segmentation	13
6.1	Dataset: TreePolygons	13
6.2	DINOv2 Segmentation Architecture	13
6.3	Segmentation Metrics	14
6.4	Segmentation Results	14
7	Focus on Low Tree Density	15
7.1	Motivation	15
7.2	Sliding-Window DINOv2 Pipeline	16
7.2.1	Input and Output Format	16
7.2.2	Sliding-Window Algorithm	16
7.2.3	DINOv2 Patch Classifier	17
7.2.4	Patch Ground-Truth Assignment	17
7.2.5	Density Map Reconstruction	18
7.3	RF-DETR Pipeline	20
7.3.1	Motivation	20
7.3.2	Relation to DETR	20
7.3.3	Input and Annotation Format	20
7.3.4	Training Setup	21
7.3.5	Output Format	21
7.4	Results	22
7.5	Discussion	23
8	Conclusion	23
9	Future Work	23

1 Introduction

Monitoring tree populations at scale is a central challenge in ecology. Ecologists study individual tree dynamics — growth, aging, spatial distribution — from aerial drone imagery. Manual annotation of these images is slow, costly, and limits the spatial and temporal scope of ecological studies. Automating tree detection and crown delineation would directly enable larger-scale, higher-frequency monitoring.

In this project, we address two sub-problems: (1) **binary classification** — determining whether a given image region contains a tree or not — and (2) **semantic segmentation** — producing a pixel-wise delineation of tree crowns. We additionally tackle **full-image detection** in low tree-density scenes, which is particularly relevant to the ecological use case where trees are spatially isolated.

Dataset We work exclusively with the **MillionTrees** benchmark [4], a large-scale, globally distributed collection of aerial imagery with three annotation formats (points, bounding boxes, polygons) spanning diverse ecosystems and imaging conditions. Other candidate datasets (IDTrees, Auto Arborist, TreeSatAI) were considered but found unsuitable for our tasks (Section 2).

Contributions Our main contributions are :

- **Patch dataset construction** We design and build a patch-based binary classification dataset from the TreeBoxes split of MillionTrees, with controlled positive/negative sampling, IoU-based filtering, and a reproducible train/test protocol.
- **Systematic model comparison** We train and evaluate five model families on the patch classification task: a CNN from scratch, a Vision Transformer (ViT) from scratch, a fine-tuned ResNet, DINOv2 with a frozen backbone and learned head, and CLIP in a zero-shot regime. We further extend this comparison with **CoOp**, a prompt learning approach for CLIP.
- **Data augmentation study** We compare three augmentation libraries and design class-specific augmentation pipelines, demonstrating consistent gains across all models, most prominently for the from-scratch ViT.
- **DINOv2-based semantic segmentation** We train a lightweight convolutional decoder on top of frozen DINOv2 patch tokens for pixel-wise tree crown delineation on TreePolygons.
- **Full-image localisation in low-density scenes** We develop a sliding-window pipeline that projects patch classifier scores into a spatial tree probability density map, and evaluate it on a curated subset of six low tree-density images. We additionally run a preliminary RF-DETR experiment on the same subset.

Key findings Foundation models substantially outperform models trained from scratch in low-data regimes. DINOv2 with a frozen backbone achieves an AUROC of 0.985 on the base dataset; after augmentation, a ViT trained from scratch reaches 0.995. Zero-shot CLIP fails on aerial imagery (AUROC 0.32), while CoOp surprisingly closes this gap very well.

2 Dataset Exploration and Selection

2.1 Candidate Datasets

At the outset of the project, three datasets were identified as potential starting points:

- **IDTrees** [1]: A data science challenge dataset providing airborne LiDAR, hyperspectral, and RGB imagery for individual tree crown delineation and species classification in temperate forests.
- **Auto Arborist** [2]: A large-scale urban street-tree dataset combining ground-level and overhead imagery with species labels, designed to study domain shift across cities.
- **TreeSatAI** [3]: A remote sensing dataset for weakly-supervised semantic segmentation of tree species, using Sentinel-2 and aerial imagery with species-level labels.

After analysis, none of these datasets proved well-suited to our task. IDTrees focuses on species classification in constrained ecosystems; Auto Arborist is urban-centric with ground-level views; TreeSatAI requires species metadata that is not aligned with our detection objective. All three lack the annotation geometry diversity (points, boxes, polygons) needed for flexible detection benchmarking.

2.2 The MillionTrees Benchmark

We selected the **MillionTrees** benchmark [4] as our primary dataset. MillionTrees is a large-scale, globally distributed benchmark for individual tree detection from aerial and satellite imagery. It aggregates data from multiple sources across diverse ecosystems and geographic regions, and provides three annotation types:

- **TreePoints**: point annotations marking tree centres (simplest).
- **TreeBoxes**: bounding box annotations around individual tree crowns.
- **TreePolygons**: polygon annotations delineating exact crown boundaries (most precise).

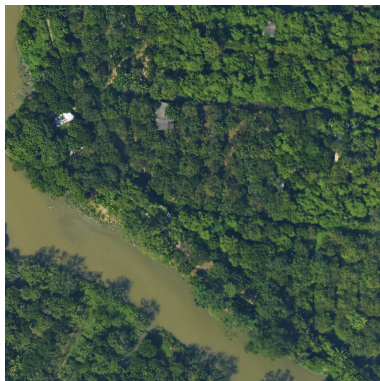
The benchmark defines multiple *split schemes* to support different evaluation tasks:

- **Random**: Standard 80/20 train/test split per source.
- **Zero-shot**: Entire data sources are held out for testing, simulating deployment on unseen domains.
- **Cross-geometry**: Mixed-geometry training (e.g., training on boxes to predict polygons).

This structure makes MillionTrees well-adapted to studying generalisation across imaging conditions, sensors, and geographic contexts — a key concern for the ecological application we target.

2.3 Data Characteristics

MillionTrees aggregates imagery from a wide range of sources: tropical forests, temperate woodlands, savanna, urban parks, agricultural land, and more. Images vary in resolution (from sub-decimeter drone imagery to meter-scale satellite data), sensor type (RGB, multispectral), and tree density. Some sources contain dense, closed-canopy forests; others contain isolated trees on open ground.



(a) Dense canopy example



(b) Sparse/savanna example



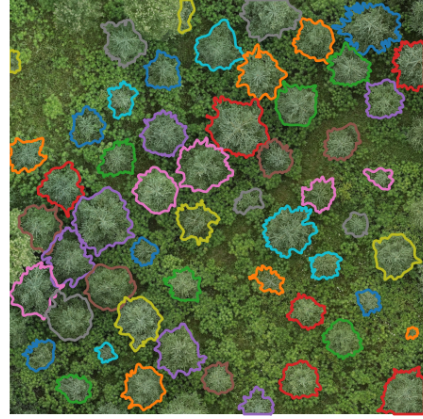
(c) Urban/isolated trees example

Figure 1: Representative aerial image examples from TreeBoxes, illustrating the diversity of ecosystems, densities, and imaging conditions present in the dataset.



(a) TreeBoxes annotations

20230712_afcahoule_itr20_p1_rgb.cog_334.png (segmented)



(b) TreePolygons annotations

Figure 2: Annotation types in MillionTrees. Left: bounding boxes over individual crowns. Right: precise polygon delineations used for segmentation.

2.4 Exploratory Data Analysis

Before building any model, we conducted an exploratory analysis of the dataset to understand the distribution of annotation sizes, class balance, and image quality. Key observations:

- Bounding box areas span several orders of magnitude, from very small boxes ($< 20\text{px}$ wide) produced in high-resolution imagery with dense planting, to large boxes ($> 3000\text{px}$) for isolated trees at lower resolution.
- Some data sources contain highly variable annotation quality, including misaligned or incomplete boxes.
- The dataset is significantly imbalanced toward positive (tree) examples in dense-canopy sources.

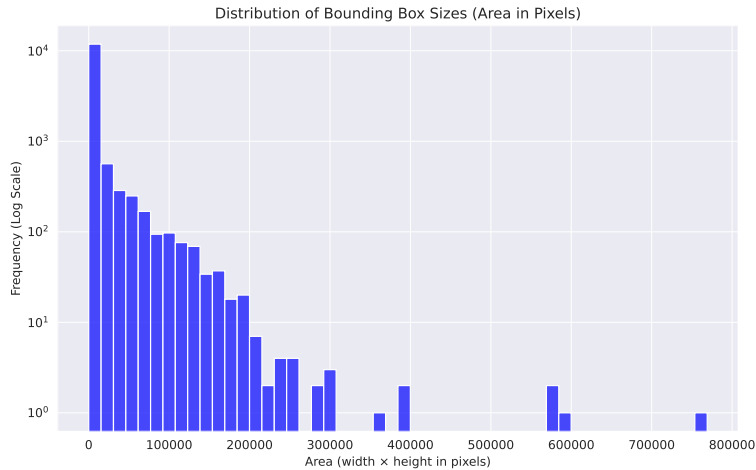


Figure 3: Distribution of bounding box sizes (width $\times$ height in pixels) across the TreeBoxes split. The large amount of small boxes compared to large ones explains the minimum-size filtering applied during patch construction.

3 Patch Dataset Construction

3.1 Rationale

We adopt a **patch-based** approach to binary classification. Rather than processing full aerial images, we extract fixed-size crops centred on candidate regions and classify each crop as tree or no-tree. This formulation is compatible with standard deep learning architectures expecting a fixed input size and makes sampling, balancing, and augmentation straightforward. All patches are resized to 224×224 pixels.

3.2 Positive Patch Generation

For each annotated bounding box in the TreeBoxes split, we generate one positive patch:

1. **Inflate:** Expand the box by 20% on each side. This provides crown context and ensures the tree is not artificially cropped at the patch border.
2. **Size filter:** Discard boxes whose inflated crop falls below a minimum area threshold. Very small boxes produce low-quality patches after resizing to 224×224 and add noise without useful signal.
3. **Resize:** Rescale the inflated crop to 224×224 via bilinear interpolation.

The resulting patches follow the ImageNet convention: the object of interest is approximately centred and occupies a substantial portion of the frame.

3.3 Negative Patch Generation

Negative patches are sampled from the same source images as the positives, subject to the following constraints:

1. Sample candidate patch centres uniformly at random across the image.
2. **IoU filter:** Accept a candidate as negative only if its IoU with every ground-truth bounding box is ≤ 0.1 . This ensures the patch does not primarily depict a tree crown. A small partial tree at the patch border is tolerated.
3. **Deduplication:** Reject candidates whose centre is within a minimum distance of an already-accepted negative centre, to avoid redundant patches.

The 0.1 IoU threshold is deliberately conservative: including some hard negatives (patches with slight tree overlap) would introduce label noise without proportional benefit.

3.4 Dataset Versions and Class Balance

We constructed two dataset versions:

Table 1: Patch dataset statistics across versions.

Version	Positives	Negatives	Total	Pos. ratio
V3 (base)	~7 000	~3 000	~10 000	70%
V4 (augmented)	~14 000	~9 000	~23 000	61%

V3 reflects the natural sampling imbalance from TreeBoxes (dense-canopy sources produce many more positive than negative crops). V4 is obtained by applying Albumentations-based augmentation (Section 5), which preferentially expands the negative class to reduce imbalance. All classification models account for the remaining imbalance via weighted loss.

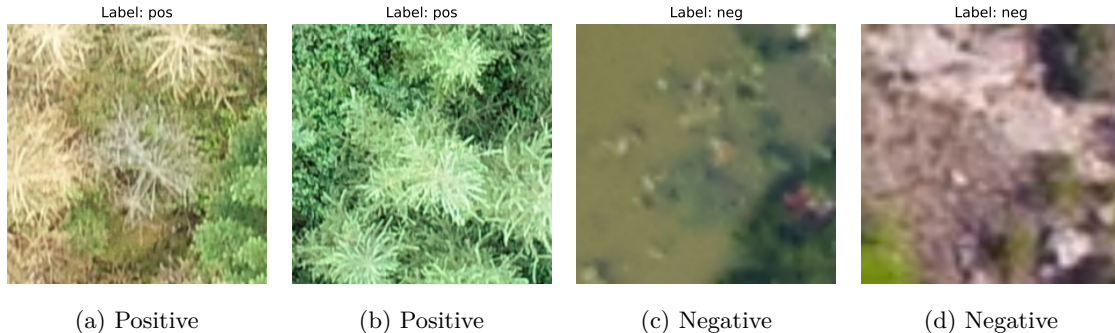


Figure 4: Example patches from V3. Positives (a, b): tree crown centred in the 224×224 crop. Negatives (c, d): background regions — soil, grass, roads, buildings — with no significant tree overlap.

3.5 Train/Test Split

The dataset is split 80/20, stratified by class label and by source image: all patches extracted from the same original aerial image are assigned to the same split. This prevents data leakage caused by highly similar adjacent crops appearing in both train and test. The split is stored as a fixed JSON manifest and shared across all experiments for comparability.

4 Binary Classification

We trained and compared five model families on the patch classification task: a lightweight CNN built from scratch, a Vision Transformer (ViT) trained from scratch, a pretrained ResNet, the DINOv2 foundation model with a learned classification head, and CLIP in a zero-shot regime.

4.1 Evaluation Protocol

All models are evaluated on the held-out test split. We report the following metrics:

- **Accuracy** and **Balanced Accuracy**: overall and class-balanced correctness.
- **Precision, Recall, F1**: standard classification metrics on the positive class.
- **AUROC**: area under the ROC curve, threshold-independent discriminative power.
- **Average Precision (AP)**: area under the precision-recall curve, most informative under class imbalance.

Models output a continuous probability score (via softmax over two logits), not a hard binary decision, so that ROC and PR curves can be computed. Threshold-specific metrics use $\tau = 0.5$ unless stated otherwise.

4.2 CNN from Scratch

We designed a simple three-block convolutional architecture as a baseline. Each block consists of:

$$\text{Conv}_{3 \times 3} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}_{2 \times 2}$$

with 16, 32, and 64 filters respectively. Padding of 1 preserves spatial resolution across convolutions; only the MaxPool reduces it. After the three blocks, a **Global Average Pooling** (GAP) layer compresses each feature map to a single value, yielding a 64-dimensional descriptor. A final **Linear**(64, 1) layer followed by sigmoid produces the output probability.

This design choice — GAP rather than flatten — significantly reduces the number of parameters in the classifier head and mitigates overfitting, at the cost of discarding fine spatial structure.

The dataset is imbalanced ($\sim 70\%$ positive). We counteract this with a **weighted binary cross-entropy loss**:

$$\text{pos_weight} = \frac{N_{\text{neg}}}{N_{\text{pos}}}$$

which penalises false negatives proportionally more. The model is trained with Adam ($\text{lr} = 10^{-3}$) for 10 epochs.

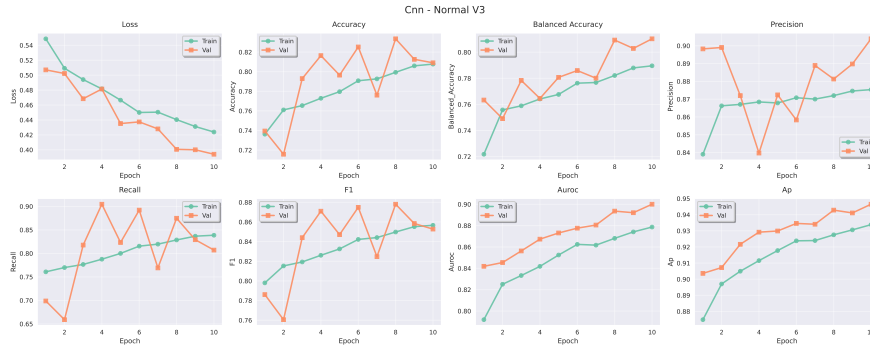


Figure 5: CNN from scratch: Training/validation loss and metrics curves on V3 test set.

4.3 Vision Transformer from Scratch

We trained a Vision Transformer (ViT) from scratch on the patch dataset, using a small-scale configuration compatible with our dataset size ($\sim 10\text{K}$ training patches). The ViT divides each 224×224 patch into a sequence of 16×16 pixel sub-patches, encodes them with positional embeddings, and processes them through transformer blocks. The [CLS] token output feeds into a two-class linear classifier.

Training a ViT from scratch on a relatively small dataset is challenging due to the inductive bias mismatch (ViTs require more data than CNNs to learn spatial structure). We used extensive data augmentation (see Section 5) to partially compensate. The model was trained with AdamW, a cosine annealing learning rate schedule, and label smoothing.

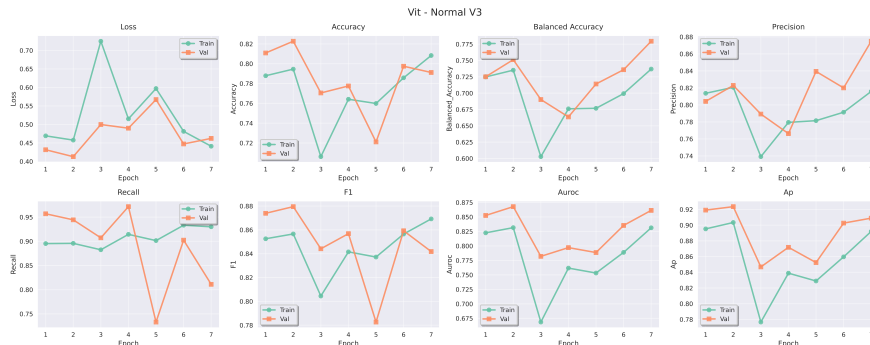


Figure 6: ViT from scratch: Training/validation loss and metrics curves on V3 test set.

4.4 ResNet (Pretrained, Fine-tuned)

We fine-tuned a pretrained ResNet (ImageNet weights) by replacing its final fully connected layer with a binary classifier (`Linear(512, 2)`). The backbone was partially frozen during the first epochs and progressively unfrozen (gradual unfreezing). The model benefits from low-level texture and edge features learned on ImageNet, which transfer reasonably well to aerial RGB patches.

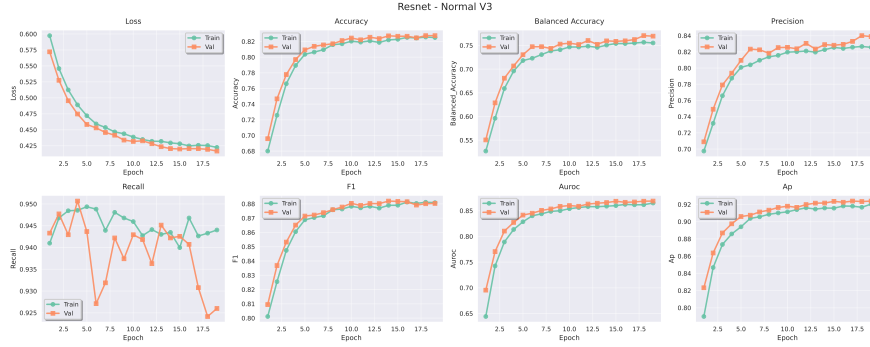


Figure 7: ResNet: Training/validation loss and metrics curves on V3 test set.

4.5 DINOv2 with Linear Head

DINOv2 [5] is a self-supervised Vision Transformer trained on 142 million images using a knowledge distillation framework without labels. It produces highly generic visual representations that transfer effectively to diverse downstream tasks.

We use the **ViT-S/14** variant (patch size 14, feature dimension 384). The backbone is **completely frozen**; only a lightweight MLP head is trained:

$$\text{Linear}(384, 128) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.2) \rightarrow \text{Linear}(128, 2)$$

The backbone is called via `backbone(x)`, which returns the [CLS] token — a single 384-dimensional vector summarising the entire patch. This is the appropriate extraction mode for patch-level classification (as opposed to pixel-level tasks, which require patch token extraction).

Class weights are recomputed dynamically from the train split to handle imbalance. Training uses Adam ($\text{lr} = 10^{-3}$), StepLR scheduler (halved every 5 epochs), and early stopping monitored on validation F1 (patience = 5).

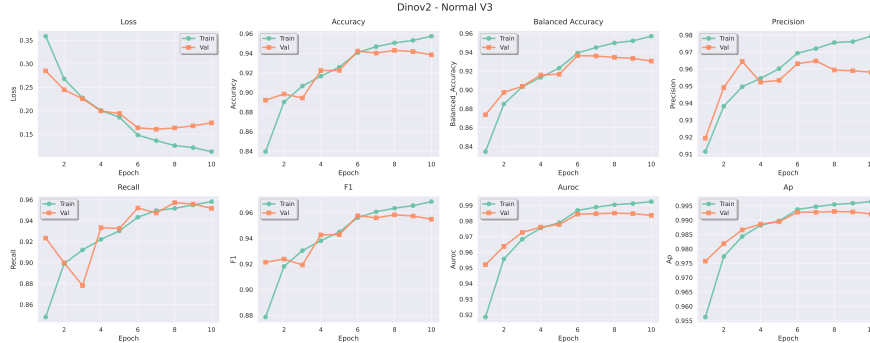


Figure 8: DINOv2 classification head: Training/validation loss and metrics curves on V3 test set.

4.6 CLIP Zero-Shot Baseline

CLIP [6] is a vision-language model trained on 400 million image-text pairs via contrastive learning. It produces a joint embedding space in which an image and a matching text description are geometrically close. At inference, zero-shot classification requires no labelled examples: given K text prompts $\{t_1, \dots, t_K\}$ describing each class, the predicted class is the one whose text embedding has the highest cosine similarity to the image embedding. This requires only a text description of the target concept — no gradient update, no labelled image — making it an interesting baseline for specialised domains where annotated data is scarce.

We tested this on our patch dataset by providing one positive prompt (describing a tree) and one negative prompt (describing a non-tree background), and assigning each patch to the class with the higher similarity score. We explored a range of prompts :

Table 2: Zero-shot CLIP performance on V3 for selected prompt pairs. Positive prompt describes a tree; negative prompt describes background. All metrics at threshold $\tau = 0.5$.

Positive prompt	Negative prompt	Prec.	Recall	F1	AUROC
a tree	not a tree	0.642	0.739	0.687	0.610
an image of a tree	an image of land without trees	0.684	0.993	0.810	0.354
aerial view of a tree crown	aerial view of empty land	0.746	0.036	0.068	0.384

The results reveal two distinct failure modes depending on prompt specificity. Generic prompts ("an image of a tree") produce very high recall (≈ 1.0): CLIP predicts tree for almost every patch regardless of content, yielding low precision and AUROC well below 0.5. It reflects that the model’s similarity scores are uninformative, assigning higher tree probability to everything including background patches.

The domain-specific prompt ("aerial view of a tree crown") partially corrects this: recall collapses to 0.036, indicating the model now almost never predicts tree. The AUROC remains low (0.384), meaning the underlying ranking is still unreliable.

CLIP’s visual encoder was trained on ground-level photographs, and aerial canopy patches — top-down texture, no trunk, no sky — lie outside this distribution. No prompt wording can compensate for this representational mismatch. Manual prompt engineering is therefore a ceiling-bounded strategy: it controls the direction of CLIP’s bias but cannot fix its inability to rank aerial patches correctly.

This motivates a **learned** adaptation of the prompt. Rather than searching over discrete word choices, we optimise the prompt context directly in embedding space using labelled patch data — keeping all CLIP weights frozen.

4.7 CoOp: Context Optimization

Context Optimization (CoOp) [9] addresses the limitations of manual prompt engineering by replacing the hand-crafted context words with **continuous learnable vectors** that are optimised end-to-end from labelled data, while all CLIP parameters remain frozen.

Formulation

In standard zero-shot CLIP, a prompt for class i takes the form:

$$t_i = [\text{context words}] [\text{CLASS}_i]$$

where the context words are hand-written tokens (e.g. "a photo of a"). CoOp replaces this fixed text with M learnable context vectors:

$$t_i = [V]_1 [V]_2 \dots [V]_M [\text{CLASS}_i]$$

Each $[V]_m \in R^{512}$ lives in the same embedding space as CLIP word tokens. These vectors are not constrained to correspond to any real word; they are free parameters optimised by gradient descent to minimise classification loss on the training set. The class token embedding is fixed (it comes from CLIP’s frozen text encoder), and the same softmax prediction rule as Equation (1) applies.

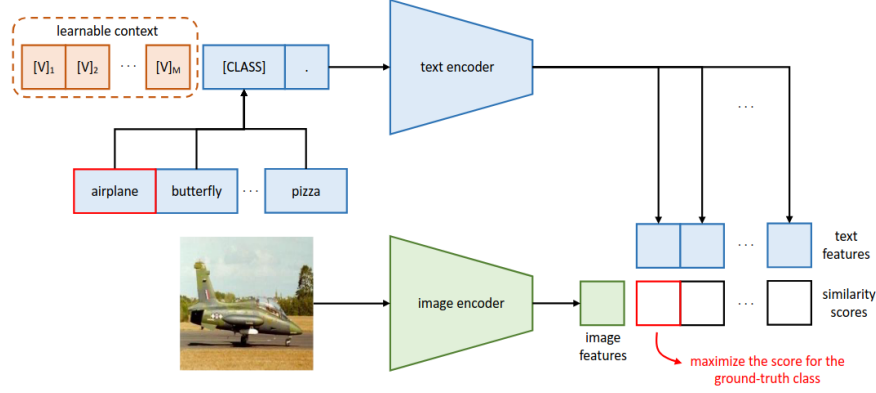


Figure 9: CoOp architecture (taken from the original paper). Orange blocks ($[V]_1 \dots [V]_M$) are the only learnable parameters. All CLIP weights are frozen. At inference, the learned context vectors replace the hand-crafted prompt, and classification proceeds identically to zero-shot CLIP.

CoOp offers two context variants and two class-token positions:

- **Unified context (UC):** A single set of context vectors $[V]_1 \dots [V]_M$ is shared across all classes. Simpler and generally better-performing.
- **Class-specific context (CSC):** Independent context vectors per class. More expressive, and shown to perform competitively on satellite and remote sensing datasets such as EuroSAT [9] — a distribution closer to our aerial imagery. We use UC as our primary setup following the paper’s recommendation, but CSC is a natural direction for future work.
- **Class token position:** End-of-sequence (as above) or mid-sequence which gives the encoder more flexibility but tends to benefit longer contexts ($[V]_1 \dots [V]_{M/2}$ [CLASS] $[V]_{M/2+1} \dots [V]_M$).

We use $M = 16$ context tokens with random initialisation. The paper shows that random and manual initialisation yield equivalent performance (both 72.65% average on the paper’s benchmark), so no domain-specific warm-start is needed. The context vectors are optimised with SGD on our patch training set; all other parameters remain frozen.

Why CoOp Is Interesting for Our Task

The appeal of CoOp is that it retains CLIP’s zero-shot deployment interface: at inference, only a text prompt (now partly encoded in learned vectors) is needed — no image-level supervision beyond what was used to train the context. For ecological applications where the target concept may shift (different species, growth stages, or sensors), CoOp offers a lightweight adaptation mechanism that does not require retraining any visual backbone.

Finally, we wanted to know if the text in [CLASS] can change the results. We tried with "tree"/"no tree" and "tree"/"background". We found slightly better results with the second option, which means that this method could still benefit from more learning. For future work, it would be interesting to have those classes learned as well.

4.8 Classification Results

Table 3: Binary classification results on **Dataset V3** (base, no augmentation). Best values in **bold**.

Model	Acc.	Bal. Acc.	Prec.	Recall	F1	AUROC
DINOv2	0.943	0.934	0.959	0.957	0.958	0.985
ViT scratch	0.930	0.900	0.930	0.960	0.948	0.970
CoOp	0.868	0.816	0.865	0.955	0.908	0.920
ResNet	0.827	0.759	0.828	0.942	0.881	0.865
CNN scratch	0.830	0.800	0.880	0.870	0.877	0.890
CLIP (zero-shot)	0.680	0.500	0.680	1.000	0.810	0.320

Table 4: Binary classification results on **Dataset V4** (augmented with Albumentations). Best values in **bold**.

Model	Acc.	Bal. Acc.	Prec.	Recall	F1	AUROC
ViT scratch	0.976	0.967	0.974	0.990	0.982	0.995
DINOv2	0.962	0.959	0.977	0.967	0.972	0.994
CoOp	0.878	0.840	0.886	0.942	0.913	0.935
CNN scratch	0.849	0.813	0.874	0.910	0.892	0.907
ResNet	0.833	0.780	0.847	0.922	0.883	0.872

Analysis

DINOv2 achieves the best results on V3, confirming the power of large-scale self-supervised pretraining even with a frozen backbone and a minimal learnable head. Only four linear layers (MLP head) are trained, yet it outperforms the fine-tuned ResNet by a wide margin. This demonstrates that DINOv2’s representations already capture discriminative visual features for aerial tree patches.

On V4 (augmented), the ViT trained from scratch surpasses DINOv2. This is notable: augmentation appears to give the scratch-trained ViT the diversity of training signal it needs to match or exceed a much larger pretrained model. Both reach near-perfect AUROC (≥ 0.994), suggesting the task is nearly linearly separable in the embedding spaces of both models when sufficient data is available.

ResNet and the CNN lag behind significantly, particularly in balanced accuracy, suggesting they are more affected by the class imbalance and the inductive bias limitations at this resolution.

CLIP zero-shot is the only model that fundamentally fails: its AUROC of 0.32 (worse than random for this framing) reflects that its ranking of positive vs. negative patches is inverted relative to our task, not merely uninformative. CoOp on the other hand does well compared to ResNet

5 Data Augmentation

5.1 Motivation

Our initial patch dataset (V3, $\sim 10K$ patches) is relatively small for training deep models robustly. Overfitting is a concern, particularly for the ViT which lacks the inductive biases of CNNs. Data augmentation artificially expands the training set by applying label-preserving transformations to existing patches.

Unlike natural image benchmarks where augmentations are standardised (horizontal flip, random crop, colour jitter), aerial patches require careful thinking:

- A tree crown remains a tree crown under rotation, horizontal/vertical flip, brightness changes, and Gaussian noise — these are safe.
- Extreme geometric distortions (very large shear, perspective warp) can displace the crown from the centre, potentially creating a misleading patch.

- Negative patches (background) are robust to most augmentations since they contain no specific semantic content.

To augment, we used **Albumentations**, a high-performance augmentation library with a clean API, fast execution (OpenCV backend), and a wide range of transforms.

5.2 Augmentation Pipeline

We designed two augmentation regimes:

Augmentation V1 (light): Conservative transforms to avoid introducing artefacts. Applied to both positive and negative patches.

- Horizontal and vertical flip ($p=0.5$ each)
- Random rotation up to 45° ($p=0.5$)
- Brightness and contrast jitter ($\pm 20\%$, $p=0.5$)
- Gaussian noise (variance up to 10, $p=0.3$)

Augmentation V2 (heavy): More varied transforms, applied more aggressively, intended to maximise diversity in the negative class in particular.

- All V1 transforms, plus:
- Random scale and shift (shift limit 10%, scale limit 20%, $p=0.5$)
- Elastic transform ($\alpha=1$, $\sigma=50$, $p=0.3$)
- Random brightness/contrast with wider range ($p=0.4$)
- CoarseDropout (random patch masking, $p=0.2$)
- Hue-saturation variation ($p=0.3$)

The choice of augmentation strategy depended on the class: positives used V1 to preserve the tree’s centre-weighted structure; negatives used a mix of V1 and V2 since they contain no semantic target that could be disrupted.

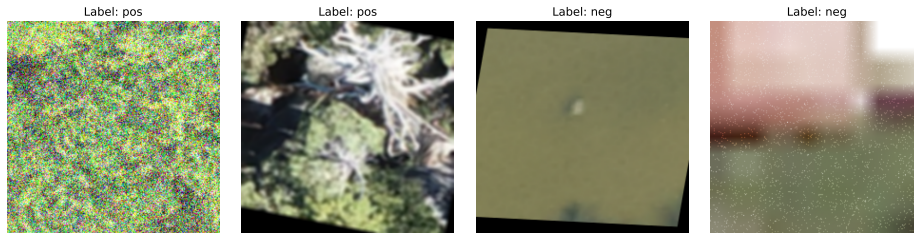


Figure 10: Augmentation examples (tree crown).

5.3 Impact of Augmentation

Comparing Tables 3 and 4, augmentation with Albumentations (V4) consistently improves all models. The most dramatic gains are observed for the ViT trained from scratch (+4.6% accuracy, +0.025 AUROC), confirming that ViTs are more data-hungry than CNNs and benefit most from increased training diversity. DINOv2 also improves, though more modestly (+1.9% accuracy), suggesting its frozen representations already generalise well on V3.

6 Semantic Segmentation

The classification task tells us *whether* a patch contains a tree. Segmentation goes further: it outputs a pixel-wise binary mask indicating *where exactly* trees are within the image. This is critical for the ecological use case, which requires accurate delineation of individual tree crowns for area estimation and growth tracking.

6.1 Dataset: TreePolygons

For segmentation, we use the **TreePolygons** split of MillionTrees, which provides precise polygon annotations for individual tree crowns. These polygons are rasterised into binary masks (0 = background, 1 = tree) at the image’s native resolution, then resized and padded to 448×448 for model input.

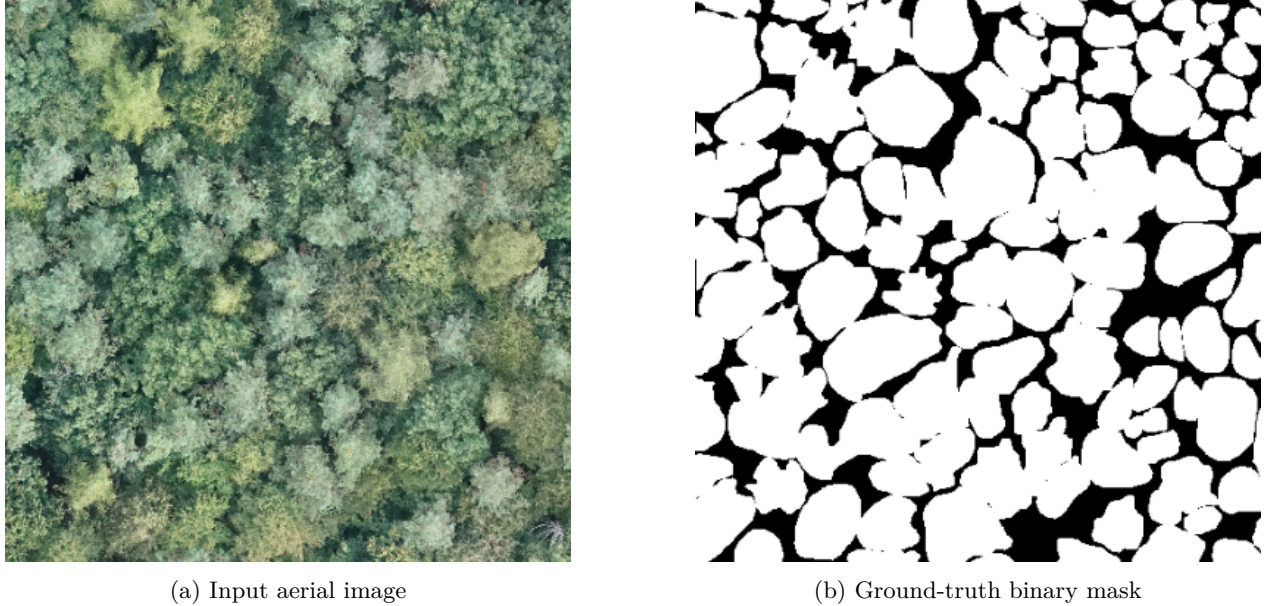


Figure 11: Segmentation training example from TreePolygons. The mask encodes individual tree crowns as connected foreground regions.

6.2 DINOv2 Segmentation Architecture

We built a segmentation model on top of the same frozen DINOv2 ViT-S/14 backbone used for classification. The key difference is how we extract and use its representations.

Feature Extraction: Patch Tokens

For classification, we used the [CLS] token — a single global summary vector. For segmentation, we need spatial information: *where* in the image are the trees? We therefore extract the **patch tokens**, which are the internal transformer representations before any classification head, giving us access to the per-patch descriptors. For a 448×448 input and patch size 14:

$$N = \frac{448}{14} \times \frac{448}{14} = 32 \times 32 = 1024 \text{ patches}$$

Each patch token is a 384-dimensional vector. The full output has shape $(B, 1024, 384)$.

Spatial Reshaping

The sequence of patch tokens is reshaped into a spatial feature map:

$$(B, 1024, 384) \xrightarrow{\text{permute} + \text{reshape}} (B, 384, 32, 32)$$

This restores the 2D spatial structure of the image, enabling convolution operations.

Convolutional Decoder

A lightweight convolutional decoder is trained on top of the fixed DINOv2 features:

$$(B, 384, 32, 32) \xrightarrow{\text{Conv}_{1 \times 1} + \text{ReLU}} (B, 128, 32, 32) \xrightarrow{\text{Conv}_{1 \times 1}} (B, 2, 32, 32)$$

This produces two logits per spatial position (background / tree). The output is upsampled back to $(B, 2, 448, 448)$ via bilinear interpolation, matching the resolution of the ground-truth mask.

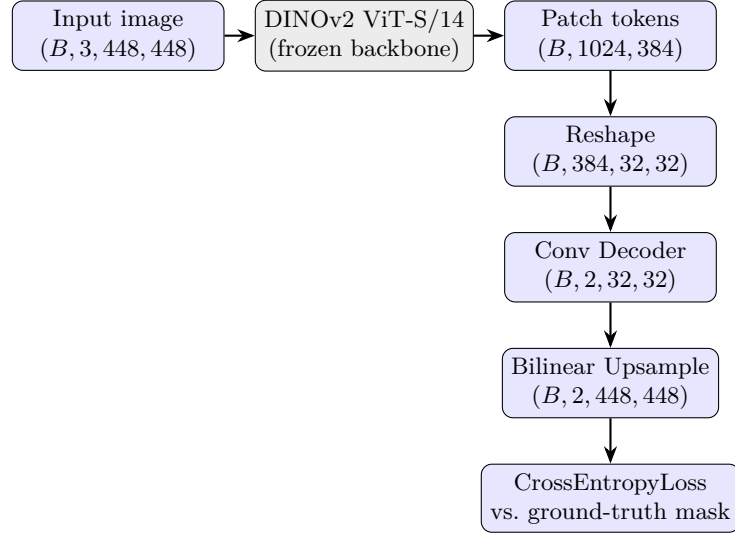


Figure 12: DINOv2 segmentation pipeline. The backbone is frozen throughout training; only the convolutional decoder learns to map patch token feature maps to binary segmentation masks.

We use cross-entropy loss over all pixels (background / tree), an Adam optimiser with $\text{lr} = 10^{-4}$.

6.3 Segmentation Metrics

We report three standard segmentation metrics:

- **Precision:** fraction of predicted tree pixels that are truly tree.
- **Recall:** fraction of true tree pixels correctly predicted.
- **IoU (Intersection over Union):** $\frac{TP}{TP+FP+FN}$, the standard segmentation metric. More stringent than either precision or recall alone.

6.4 Segmentation Results

Table 5: Segmentation results on TreePolygons test set.

Model	Precision	Recall	IoU
DINOv2 (frozen backbone + decoder)	0.769	0.950	0.738

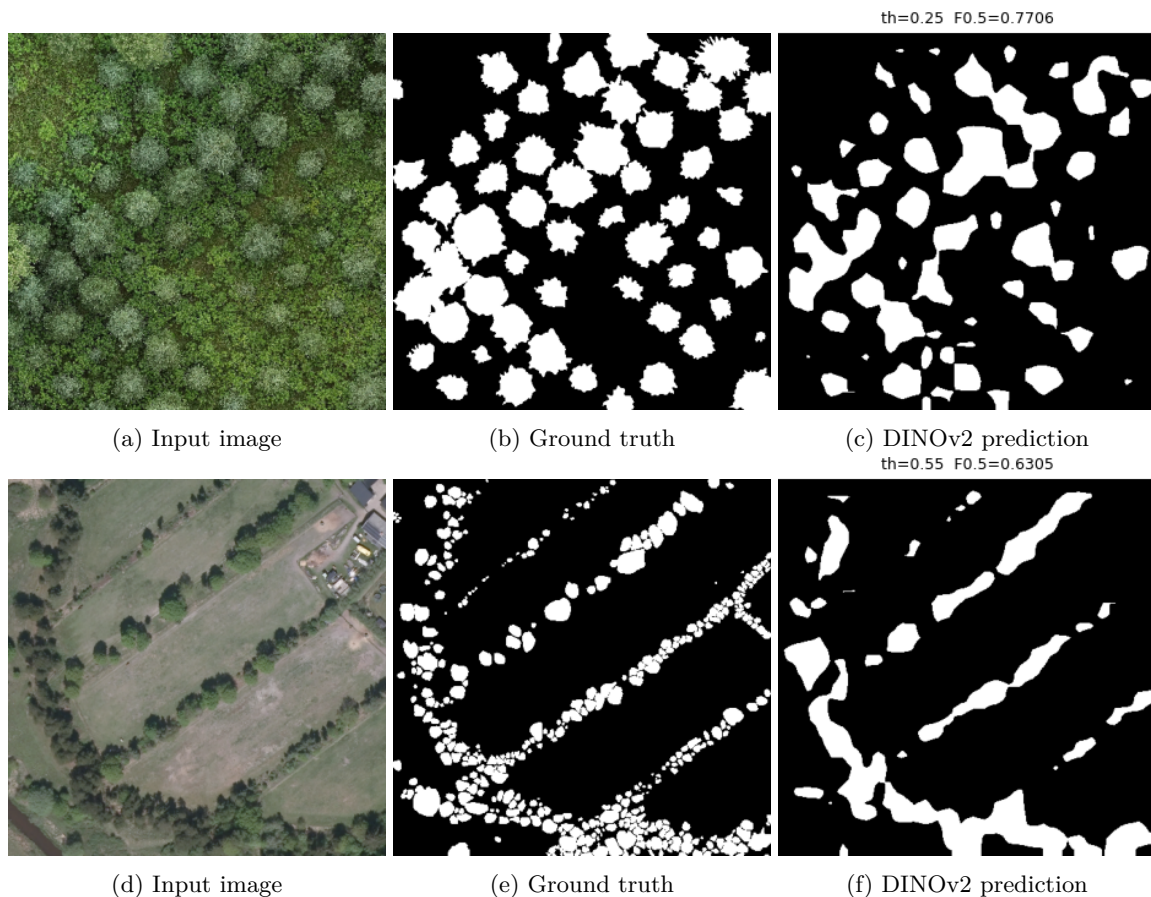


Figure 13: Qualitative segmentation results. The model correctly identifies most tree regions (high recall = 0.950) but sometimes includes adjacent non-tree pixels (precision = 0.769).

Analysis

The DINOv2 segmentation model achieves a recall of 0.950, meaning it identifies nearly all true tree pixels. Precision is lower at 0.769, indicating some over-segmentation: the model tends to label adjacent background pixels (often the ground or shadow directly beneath/around crowns) as tree. The IoU of 0.738 reflects this bias.

This behaviour is partly inherent to the architecture: the patch-token feature map has resolution 32×32 before upsampling to 448×448 — a factor-of-14 interpolation that cannot recover fine boundary details. The decoder’s 1×1 convolutions cannot reason about sub-patch spatial structure. This is a fundamental limitation of the approach that could be addressed by using higher-resolution models or a multi-scale decoder. Increasing the decision threshold (from 0.5 toward 0.7–0.8) improves precision at the cost of recall, which may be preferable depending on the downstream use (precise crown boundaries vs. coverage estimation).

7 Focus on Low Tree Density

7.1 Motivation

The goal of this part is to study whether the behaviour of the models depends on the **tree density** of the images. From the annotated mini dataset, we isolated the **six low-density images**, corresponding to images where trees are more sparse and spatially separated than in the rest of the dataset.



Figure 14: Low density image example

These six images are treated as a dedicated low-density evaluation subset. They are kept aside from the RF-DETR training data and are used only for inference and evaluation.

The low-density images do not all have the same resolution. Three images have size 400×400 , while the three tile images have size 640×640 . For this reason, the low-density experiments are defined relative to each image size, rather than assuming a fixed image resolution.

We evaluate two complementary strategies:

- **Sliding-window DINOv2**: a patch classifier is applied over the full image to build a tree-probability density map.
- **RF-DETR**: an object detector is trained to predict tree bounding boxes directly on the full image.

The sliding-window approach gives a spatial probability map, while RF-DETR gives explicit tree detections.

7.2 Sliding-Window DINOv2 Pipeline

7.2.1 Input and Output Format

The input of this pipeline is a full RGB aerial image from the low-density subset, together with its tree bounding-box annotations.

The input images have either size 400×400 or 640×640 . They are not directly resized as full images. Instead, each image is scanned with square patches.

Each extracted patch is resized to the classifier input size, 224×224 , before being passed through the DINOv2 binary classifier.

The output of the model for each patch is:

- a **tree probability**, obtained from the softmax score of the positive class;
- a **binary patch prediction**, obtained by thresholding the probability at 0.5.

The final output of the sliding-window pipeline is not a bounding box prediction. It is a **tree-probability density map** projected back onto the original image.

7.2.2 Sliding-Window Algorithm

The DINOv2 classifier was trained to classify individual image patches. To apply it to a full image, we use a sliding-window strategy.

For each low-density image, we define a nominal grid size and derive a corresponding patch size from the image dimensions. We tested several grid granularities. The initial coarse 4×4 setting was later discarded from the main analysis because the patches were too large: in these low-density images, most large patches still contained at least part of a tree, making the evaluation less informative.

The main analysis therefore focuses on finer patch decompositions:

- 8×8 **nominal grid**

- 16×16 **nominal grid**
- 32×32 **nominal grid**

For each nominal grid, the window is moved with a **50% stride**, meaning that neighbouring patches overlap by half of the patch size. This increases spatial coverage and reduces the risk of missing trees located near patch borders.

Because of this overlap, the number of evaluated patches is larger than the nominal grid size:

- 8×8 **grid**: 15×15 sliding windows, i.e. **225 patches per image**.
- 16×16 **grid**: 31×31 sliding windows, i.e. **961 patches per image**.
- 32×32 **grid**: 63×63 sliding windows, i.e. **3969 patches per image**.

Each patch is then processed independently by the binary DINOv2 classifier.

DINO density maps - grid 16 - bbox intersection 60% - MEAN aggregation

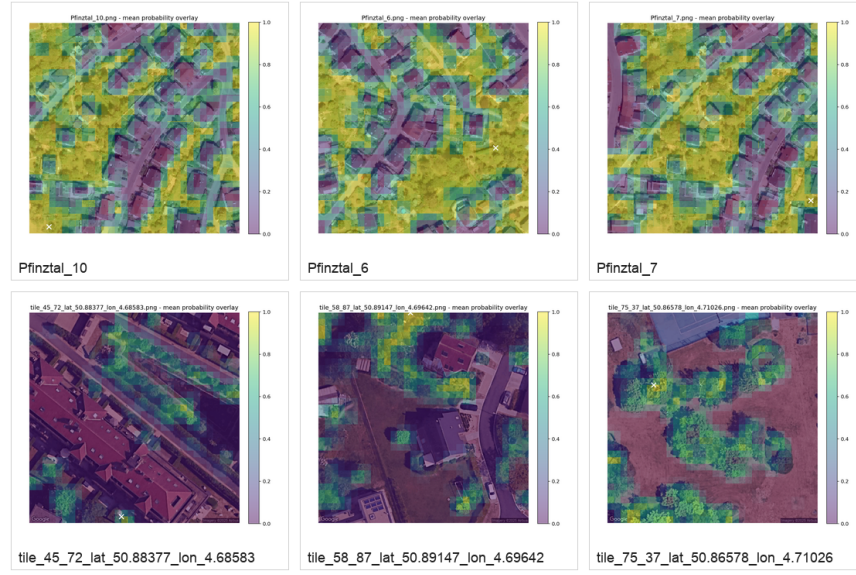


Figure 15: Density map grid 16x16, 60% intersection

7.2.3 DINOv2 Patch Classifier

The classifier uses a frozen DINOv2 ViT-S/14 backbone followed by a small binary classification head. The backbone extracts a feature vector from the resized patch, and the classification head predicts whether the patch contains a tree.

The classification head has the following structure:

Linear(384, 128) -> ReLU -> Dropout(0.2) -> Linear(128, 2)

For each patch, we keep the probability of the positive class, interpreted as the probability that the patch contains a tree.

7.2.4 Patch Ground-Truth Assignment

To evaluate the classifier at patch level, each sliding-window patch must be assigned a ground-truth label. This is done by comparing the patch window with the annotated tree bounding boxes. A patch is considered positive if it covers a sufficient fraction of at least one tree bounding box.

We tested three coverage thresholds:

- **20%**: permissive criterion; a patch is positive even if it covers only a small part of a tree box.

- **40%:** intermediate criterion.
- **60%:** stricter criterion; the patch must cover most of a tree box to be considered positive.

This allows us to check whether the evaluation is sensitive to the definition of a positive patch. For each patch size and each threshold, we compare the predicted patch label with the assigned ground-truth patch label. We report standard classification metrics, especially **recall**, because missing tree-containing regions is the main risk in low-density images.

7.2.5 Density Map Reconstruction

After inference, each patch has a tree probability. These patch probabilities are projected back onto the original image at the spatial location of the corresponding patch.

Since patches overlap, a pixel can be covered by several patches and therefore receive several probability values. We tested two ways to aggregate these values:

- **Max aggregation:** each pixel receives the highest probability among all patches covering it.
- **Mean aggregation:** each pixel receives the average probability of all patches covering it.

The **max** aggregation highlights the strongest local tree response. It is useful when we want to avoid missing potential tree regions. The **mean** aggregation produces smoother maps, but can dilute high-confidence responses when neighbouring overlapping patches have lower scores.

The resulting density map is displayed over the original image. The ground-truth bounding boxes are also shown in red. This makes it possible to visually compare the model probability map with the annotated tree locations.

DINO density maps - grid 32 - bbox intersection 60% - MEAN aggregation

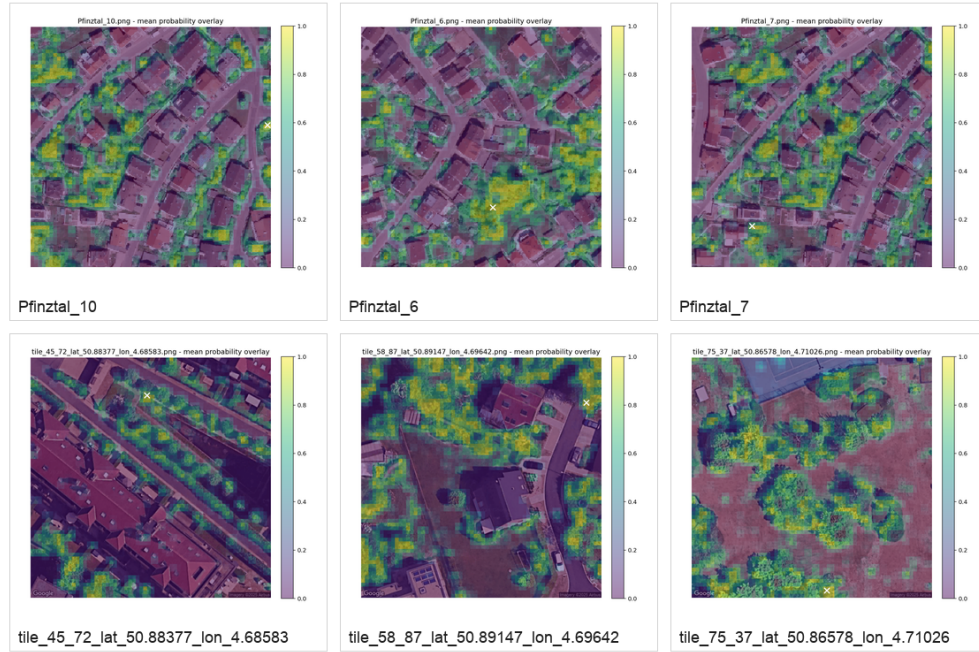


Figure 16: Density map grid 32x32, 60% intersection, Mean aggregation

DINO density maps - grid 32 - bbox intersection 60% - MAX aggregation

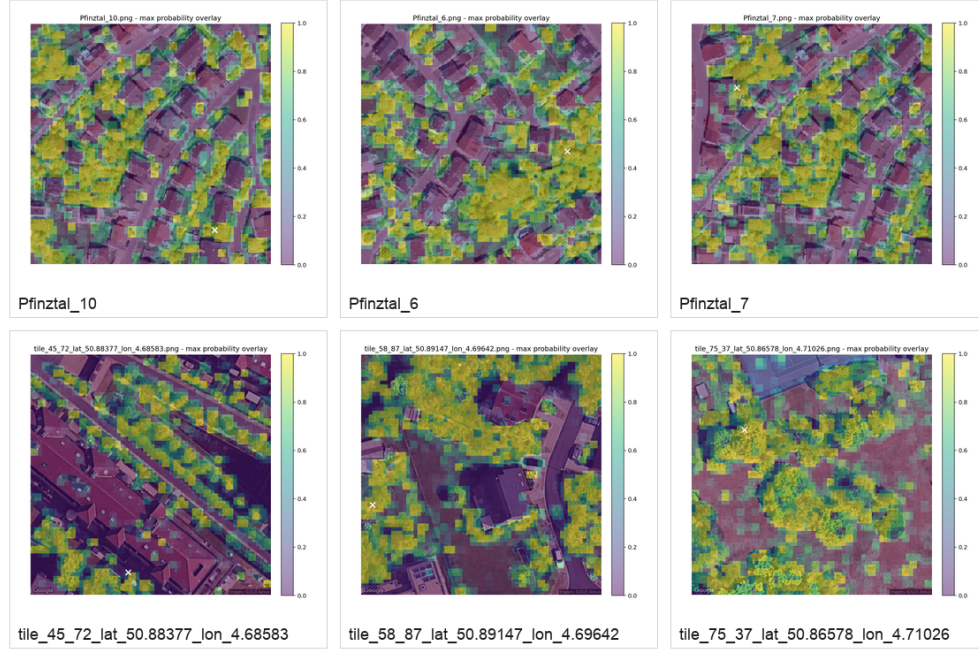


Figure 17: Density map grid 32x32, 60% intersection, Max aggregation

Table 6: Sliding-window evaluation results for the DINOv2 binary classifier on low-density images. Results are reported for different grid sizes and bbox-overlap thresholds. Best recall is shown in **bold**.

BBox overlap	Grid	Patches/img	Prec.	Recall	F1	AUROC	AP
20%	8x8	225	0.910	0.609	0.730	0.848	0.904
20%	16x16	961	0.772	0.712	0.741	0.883	0.843
20%	32x32	3969	0.446	0.508	0.475	0.796	0.442
40%	8x8	225	0.822	0.607	0.698	0.791	0.831
40%	16x16	961	0.697	0.770	0.732	0.891	0.820
40%	32x32	3969	0.370	0.552	0.443	0.809	0.386
60%	8x8	225	0.738	0.597	0.660	0.746	0.743
60%	16x16	961	0.632	0.805	0.708	0.896	0.796
60%	32x32	3969	0.301	0.587	0.398	0.814	0.317

Overall, the best recall is obtained with the 16x16 grid and a 60% bbox-overlap threshold, suggesting that an intermediate patch size provides the best compromise between spatial precision and sufficient visual context for tree presence classification.

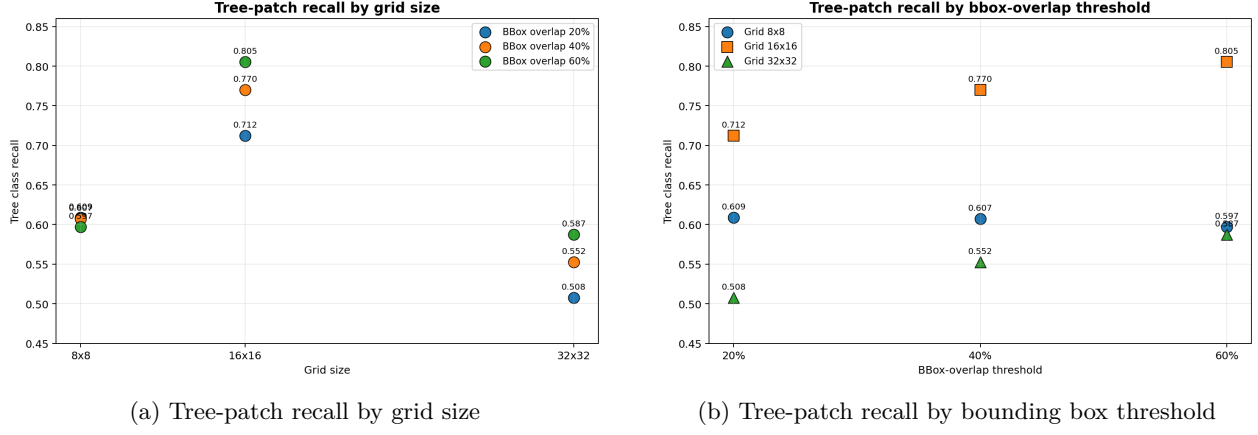


Figure 18: Comparison of tree-patch recall across sliding-window grid sizes and bounding-box overlap thresholds.

7.3 RF-DETR Pipeline

7.3.1 Motivation

The sliding-window method produces a probability map, but it does not directly predict individual tree bounding boxes. To complement this approach, we also use **RF-DETR**, an object detector that predicts tree bounding boxes directly from the full image.

7.3.2 Relation to DETR

The original DETR model views object detection as a direct set prediction problem. Instead of scanning the image with patches or relying on manually designed anchors, DETR predicts a fixed set of possible objects from the full image.

Conceptually, DETR follows this structure:

image $\rightarrow$ visual backbone $\rightarrow$ transformer encoder/decoder $\rightarrow$ object queries $\rightarrow$ boxes and classes

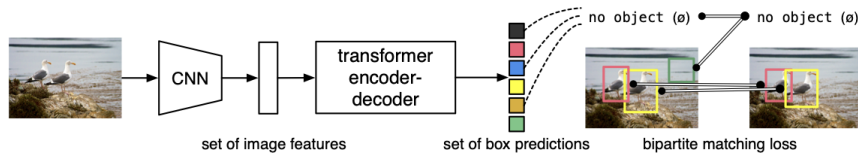


Figure 19: DETR architecture

Each object query acts as a detection slot. A slot can either predict an object or predict no object. During training, predicted boxes are matched with ground-truth boxes so that each annotated object is assigned to one prediction.

RF-DETR follows the same general detection philosophy, but provides a more practical implementation for custom datasets. In our case, it is used with a single object class: **tree**.

7.3.3 Input and Annotation Format

The input of RF-DETR is a full RGB aerial image. Unlike the sliding-window approach, the image is not decomposed into manually defined patches before inference. The detector processes the full image and directly predicts tree boxes.

The original bounding-box annotations are stored as:

filename, xmin, ymin, xmax, ymax

For RF-DETR training, these annotations are converted to COCO detection format. In COCO, each image is described by:

`id, file_name, width, height`

Each annotation is described by:

`id, image_id, category_id, bbox, area, iscrowd`

The bounding box in COCO format is:

`bbox = [xmin, ymin, width, height]`

Thus, each original annotation is converted from:

`[xmin, ymin, xmax, ymax]`

to:

`[xmin, ymin, xmax - xmin, ymax - ymin]`

The category list contains a single class:

`tree`

7.3.4 Training Setup

For the RF-DETR experiment, the six low-density images are explicitly excluded from the training and validation data. This prevents data leakage and ensures that the low-density subset is used only for inference and evaluation.

After removing the low-density images, the training data contains:

- **17 training images** with **7,087 valid bounding boxes**;
- **9 validation images** with **3,081 valid bounding boxes**.

We use **RFDETRMedium** with an input resolution of 640 pixels. This model size was chosen as a compromise between capacity and computational cost.

7.3.5 Output Format

At inference time, RF-DETR outputs a set of detections for each image. Each detection contains:

- the predicted bounding box in `xmin, ymin, xmax, ymax` format;
- a confidence score;
- the predicted class, here **tree**.

The RF-DETR output can therefore be compared with the ground-truth tree boxes. This provides a complementary evaluation to the sliding-window density maps: DINOv2 produces a probability field over the image, while RF-DETR produces discrete object detections.

7.4 Results

The results below show the bounding box on the images. PURPLE boxes are false positive from DETR, RED are missed and GREEN are good predictions. The bounding box is considered coinciding with the ground truth if the are of intersection is of at least 30%.

RF-DETR bbox matching overlays - IoU ≥ 0.30

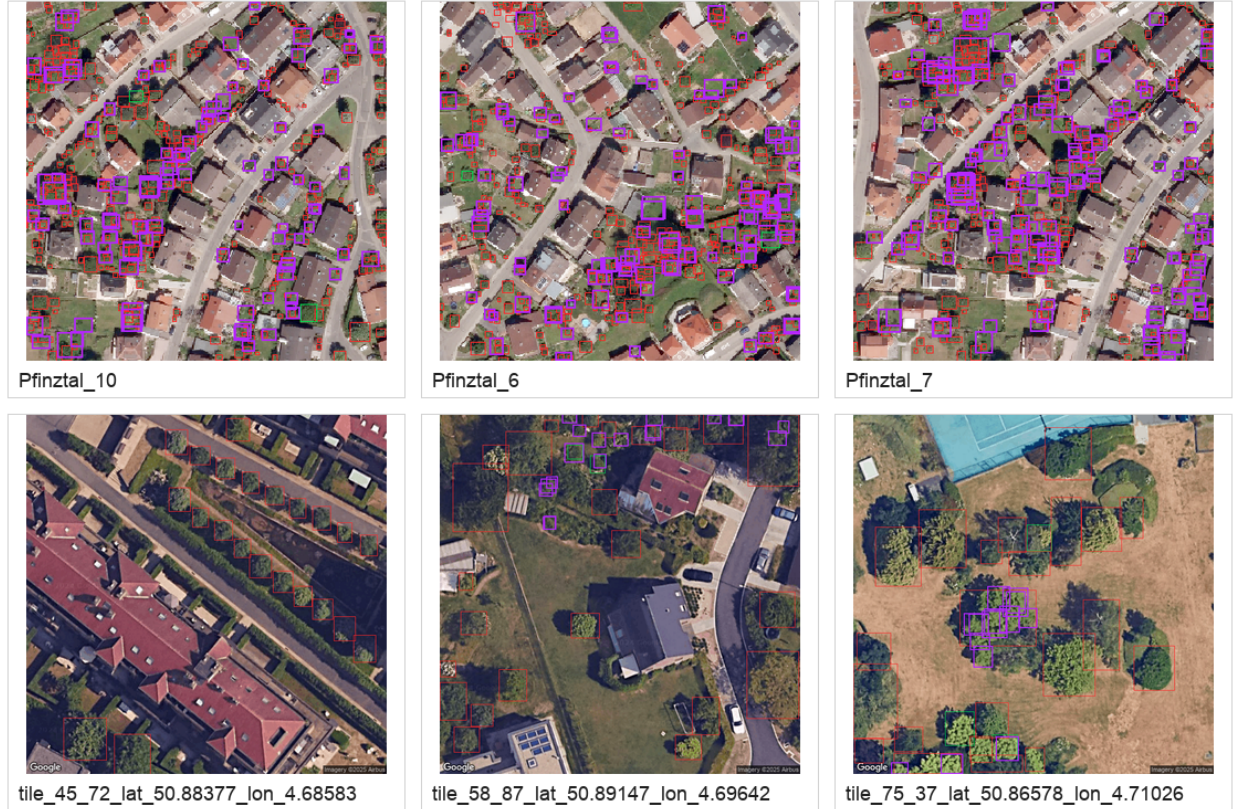


Figure 20: RF-DETR detections on low-density images matched to ground truth boxes

The RF-DETR experiment should be interpreted as a preliminary attempt rather than a fully optimized detection pipeline. The model was fine-tuned on a very limited number of training images, which strongly restricts its ability to generalize to the low-density test images. In this preliminary RF-DETR experiment, the six low-density images were kept exclusively for inference. The remaining TreeBoxes subset contained only 26 annotated images, split into 17 training images and 9 validation images. Although these images contain many tree bounding boxes, the number of distinct scenes is very small, which limits the diversity of backgrounds, canopy shapes, tree densities, and image acquisition conditions seen during fine-tuning so the model sees limited variation in background, lighting, canopy shape, scale, and spatial layout.

Another limitation is the use of a single generic class, *tree*. This is appropriate for a first binary detection setup, but it does not explicitly teach the model how to distinguish tree crowns from visually similar structures such as shrubs, shadows, grass, or other vegetation. Adding more classes would only be useful if these confusing objects were also annotated with bounding boxes. Without such annotations, increasing the number of labels would not improve the model.

Therefore, the poor RF-DETR results are mainly explained by the small and weakly diverse fine-tuning set, the difficulty of detecting many small aerial tree crowns, and the limited semantic information provided by a single *tree* label. The experiment remains useful as a proof of concept, and better results would likely require more training images, stronger data augmentation, and possibly additional annotated classes for common sources of confusion.

7.5 Discussion

The sliding window approach demonstrates that our patch classifiers generalise from centred, isolated patches to full images. The density maps produce visually coherent probability landscapes: high-density forest patches score uniformly high; open ground scores near zero; isolated trees produce clear isolated peaks.

Challenges include:

- **Scale sensitivity:** A fixed 224×224 window at single scale misses very small trees (only a few pixels wide) and poorly segments very large trees. Multi-scale sliding would address this but multiplies inference cost.
- **Threshold sensitivity:** Detection threshold τ must be tuned per domain; a value calibrated on one ecosystem does not transfer directly to another.
- **Low-density scenes:** In images with few trees (urban scenes, open land), the model maintains good recall but false positives become more frequent. This motivates further evaluation on manually selected low-density images.

8 Conclusion

This project investigated automatic tree detection and segmentation from aerial imagery using the Million-Trees benchmark. We constructed a patch-based binary classification dataset from TreeBoxes, evaluated five model families, studied the impact of data augmentation, extended patch classifiers to full images via sliding windows, trained a DINOv2-based semantic segmentation model, and conducted a preliminary RF-DETR object detection experiment on low-density scenes.

Foundation models consistently outperform models trained from scratch in low-data regimes. DINOv2 with a frozen backbone achieves AUROC = 0.985 on V3; a ViT trained from scratch reaches 0.995 after augmentation. CoOp substantially closes the gap left by zero-shot CLIP (AUROC 0.32 $\rightarrow$ 0.920), confirming that learned prompt adaptation is a viable strategy for domain-shifted vision-language models. For segmentation, DINOv2 patch tokens with a lightweight convolutional decoder achieve IoU = 0.738 and recall = 0.950. The sliding-window pipeline produces interpretable density maps with clear local maxima at tree locations, demonstrating that patch classifiers generalise to full-image detection without architectural changes.

9 Future Work

Several directions emerge naturally from this work:

- **Higher-resolution segmentation.** The 32×32 DINOv2 feature map limits boundary precision. A multi-scale decoder (FPN-style) or a finer-patch backbone (ViT-B/8) would improve crown delineation.
- **Instance segmentation.** Our decoder produces semantic masks; separating individual crowns is required for per-tree ecological tracking. SAM, prompted with classifier detections, is a natural next step.
- **Multi-scale sliding window.** A fixed 224×224 window fails on very small or very large trees. A feature-pyramid or scale-aware sliding window would improve recall across canopy sizes.
- **Stronger RF-DETR fine-tuning.** The RF-DETR experiment was limited by the small and low-diversity training set (17 images). More annotated scenes, stronger augmentation, and additional negative classes (shrubs, shadows) would substantially improve detection quality.
- **Zero-shot generalisation.** Evaluating on the MillionTrees zero-shot split — where entire data sources are held out — would quantify domain transfer, critical for operational ecological deployment.
- **CoOp class-specific context and learned class tokens.** CSC (class-specific context) and jointly optimising the class token embeddings are natural extensions that may further close the CLIP domain gap.

References

- [1] Marconi, S., Graves, S. J., Gong, D., Shahriari Nia, M., Le Bras, M., Dorr, B. J., ... & White, E. P. (2019). A data science challenge for converting airborne remote sensing data into ecological information. *PeerJ*, 6, e5843.
- [2] Beery, S., Wu, G., Edwards, T., Pavetic, F., Majewski, B., Mukherjee, S., ... & Huang, J. (2022). The Auto Arborist Dataset: A large-scale benchmark for multiview urban forest monitoring under domain shift. In *CVPR 2022* (pp. 21294–21307).
- [3] Ahlswede, S., Thekke-Madam, N., Schulz, C., Kleinschmit, B., & Demir, B. (2022). Weakly supervised semantic segmentation of remote sensing images for tree species classification based on explanation methods. *arXiv:2201.07495*.
- [4] Weinstein, B. G., Marconi, S., Bohlman, S. A., Magee, L., & White, E. P. (2023). MillionTrees: A large-scale benchmark for individual tree detection from aerial imagery. *arXiv preprint*.
- [5] Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., ... & Bourdoukan, R. (2023). DINOv2: Learning robust visual features without supervision. *arXiv:2304.07193*.
- [6] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In *ICML 2021*.
- [7] Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-end object detection with transformers. In *ECCV 2020*.
- [8] Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., ... & Girshick, R. (2023). Segment anything. In *ICCV 2023*.
- [9] Zhou, K., Yang, J., Loy, C. C., & Liu, Z. (2022). Learning to prompt for vision-language models. *arXiv:2109.01134*.